# Paso 2 — Vocabulario: ¿existe el bloque? Si no, se crea y se instala

**Fecha:** 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 2 de `docs/AgentPDFs/WORKFLOW.md` (fila «Vocabulario»): `sqx/inspect/vocabulary.py`, `sqx/blocks/install.py`, `sqx/blocks/taxonomy.yaml` (y los dos ficheros que lo leen: `taxonomy.py`, `palette.py`), las dos skills `sqx-custom-block` y `sqx-random-group`, `sqx/CLAUDE.md` y las fichas de `knowhow/authoring/`.

**Qué es:** el documento de consulta del paso 2. Este paso **no juzga ni calcula estadística**: decide si una idea ya se puede expresar con lo que SQX trae, y si no, deja escrito el bloque en las instalaciones que construyen. Por eso no hay «criterio cargado» en `criteria.yaml` ni umbral en `ledger/thresholds.yaml`; lo que hay son comprobaciones y negativas del código, y las repaso todas.

**De dónde salen los números:** recalculados por mí el 2026-10-08 leyendo los XML vivos de `SQX_w1`, `SQX_w2` y `SQX` (sólo lectura, sin arrancar SQX) con las propias funciones del módulo; el script está en `/tmp/claude-1000/-home-sergioguslw-Desktop-AlgoProject/7f3c5eea-c6b2-4ac8-99b3-5bc4a3b2196f/scratchpad/paso02/recuento.py`. El ejemplo de principio a fin es el bloque `CBlock_CloseCrossesAboveKCUpper` (plantilla `keltnerUpperCrossUp`, hoy retirada a `AlgoData/templates/_retiradas-2026-10-05/library/`). No hay «corrida» de este paso: no escribe parquets, escribe XML en las instalaciones.

**Commit que describo:** `b2b4c8c`. **Sin confirmar en las carpetas del paso:** `sqx/blocks/taxonomy.yaml` modificado (8 líneas: pesos de `CSSAMarketRegime*` y de cuatro bloques `TTMSqueeze*`, y `CBlock_CSSARegimeAbove50/Below50`); `sqx/blocks/palettes/` con nueve paletas nuevas sin seguir (`*_v3`, `doubleROCLong_USDJPY_M30`, `indicesIBS_USA500_H1`, `squeezeBBLong_XAUUSD_H1`); `knowhow/authoring/INDEX.md` y `headless-authoring-chain.md` modificados (sólo un enlace `see:`). `sqx/inspect/vocabulary.py`, `sqx/blocks/install.py` y `tools/sqx-lab/` **sin cambios**. Las skills `sqx-custom-block` y `sqx-random-group` **no** están en `.claude/skills/` (de las skills `sqx-*`, allí sólo existe `sqx-wait`; la carpeta tiene 36 skills en total): viven en `tools/sqx-lab/plugins/sqx-lab/skills/` y `~/.claude/skills/` tiene enlaces a ellas (el instalador `bin/sqx-lab-install.sh` los crea y les pega encima el texto de `tools/sqx-lab/overlays/`).

**Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.**

**Ninguna línea de este documento está congelada.** Lo que aquí figura como «hoy» es lo que el código hace hoy.

---

## 0. En una página

**La pregunta del paso:** «la idea que el dueño describió, ¿existe ya como bloque que SQX pueda usar? Y si no, ¿cómo queda escrita, comprobada e instalada para que el constructor la alcance?»

**Lo cargado hoy:** nada que decida. Hay tres cosas mecánicas:

1. **Una consulta de sólo lectura** ( `python3 -m sqx.inspect.vocabulary [TÉRMINO] [--diff ROL]` ) que lee tres XML de una instalación y dice qué bloques y grupos tiene.
2. **Una comprobación del autor** (la del `validate.py` de la skill, 7 comprobaciones para bloques, 8 para grupos) que mira si el XML se puede importar. No mira si la idea es buena ni si cumple la regla 15.
3. **Un instalador** ( `python3 -m sqx.blocks.install` ) que añade o reemplaza entradas en `customBlocks.xml` / `blockGroups.xml` de un worker parado, con copia de seguridad. **No valida nada**; se fía de lo que le dé la skill.

**Lo que sale hoy en el conductor ( `SQX_w1` ):** 849 bloques nativos + 179 propios (177 de condición y 2 de nivel de precio), 25 grupos (21 de condición, de los que uno, `RandConditions` , está vacío; 4 de valor). El custodio ( `SQX_w2` ) es idéntico. El maestro ( `SQX` ) tiene 171 propios y 22 grupos: le faltan 8 bloques y 3 grupos, a propósito (nunca se escribe en él).

**Las nueve cosas que conviene saber antes de leer el resto:**

1. **«¿Existe?» lo contesta una búsqueda de subcadena sobre la clave y el texto de pantalla, nada más** ( `sqx/inspect/vocabulary.py:124` ). Buscar `keltner` devuelve **18 bloques nativos y ningún propio**, aunque el dueño tiene dos propios de Keltner: su texto de pantalla es `CloseCrossesAboveKCUpper` , sin la palabra. Con `KCUpper` sí aparece. Una respuesta «no existe» de esa búsqueda no prueba que no exista (apartado 5.1).
2. **El mensaje «NO GROUP POOLS IT — unusable in a template» sale en 808 de los 849 nativos** (los otros 41 están en un grupo de valor: `GroupMAs` , `RandomPrices` , `BollingerBands_Lower` , `BreakoutRand` ) **y en 130 de los 179 propios**, y es engañoso: un hueco libre de la plantilla sortea toda la paleta de condiciones, incluidos los nativos ( `knowhow/authoring/holes-groups-randomcondition.md` ). Sólo es cierto para un bloque que se quiere meter **en un hueco atado a un grupo**.
3. **Instalar no valida.** `sqx/blocks/install.py` acepta cualquier XML cuyo primer hijo sea `<Item>` o `<Group>` . La validación vive en las skills y es una orden aparte que hay que acordarse de correr.
4. **La regla 15 (nada de reloj, nada de entradas stop/limit) no se aplica en este paso.** Se puede escribir e instalar un bloque de reloj sin que nada proteste. La negativa está en el paso 6, al aplicar la paleta ( `sqx/projects/buildingblocks.py:62` ), y su lista de «bloques de reloj» es una expresión regular que **no atrapa** `CBlock_TimeBreakoutLong` (propio) ni `SessionHigh/Low/Open` (nativos) — y la paleta antigua `ruptura_base` los enciende (apartado 5.3).
5. **Un bloque queda donde se instala: `SQX_w1` y `SQX_w2` , nunca el maestro.** Las dos instalaciones deben quedar idénticas; `--diff custodian` comprueba **sólo un sentido** (lo que falta en el destino, no lo que sobra) y sólo por nombre. Son dos órdenes separadas sin transacción: nada impide que el conductor quede instalado y el custodio no. Si no, una plantilla «desaparece del constructor sin error».
6. **El segundo control de calidad de la skill ( `assess.py` , «higiene de ventaja») no miró el bloque del ejemplo**: imprime «no custom blocks found» y sale con código 0 incluso con `--strict` , porque sólo califica bloques con `categoryType="Custom blocks"` (apartado 5.2).
7. **Un bloque sólo queda probado cuando una construcción lo lleva.** Ni la comprobación del XML ni la instalación lo prueban; en el ejemplo, la prueba fueron 30 estrategias generadas en 29 s con el bloque en las 30 ( `manifest.json` , apartado 3.9).

8. **Terminar el paso 2 con un bloque instalado no significa que el constructor pueda sortearlo.** La taxonomía (767 filas: 596 nativas + 171 propias) va **8 propios por detrás** de los 179 del conductor, y los 8 que faltan son exactamente los que el maestro tampoco tiene, **incluido el del ejemplo** `CBlock_CloseCrossesAboveKCUpper`. Al aplicar una paleta, una clave que la taxonomía no lista se cuenta como «outside» y se deja apagada (`sqx/projects/buildingblocks.py:45-46,60`). Hasta regenerar y etiquetar (`python3 -m sqx.blocks.taxonomy`), un hueco libre no los alcanza por esa vía (calculado por mí; el efecto sobre una construcción real no lo medí).

9. **La regla 15 es ciega a todo bloque propio, y no mira lo que no pasa por una paleta.** La regex solo reconoce nombres nativos (empieza por `Current|Bar`), así que ningún `CBlock_*` puede salir clasificado como reloj, se llame como se llame. Y un bloque de reloj dentro de un grupo o fijo en la plantilla no pasa por la negativa del paso 6, que solo actúa sobre las entradas de `<BuildingBlocks>` que una paleta enciende.

---

## 1. Qué es el paso y cómo decide

### 1.1 Dónde está

Es el segundo de los 26 pasos: después de que la idea está escrita sin ambigüedades (paso 1, el `ideaExpert`) y antes de la plantilla (paso 3, `sqx-strategy-template`). Lo ejecuta el agente `templateArchitect` o la skill `sqx-strategy-template`, que llama a `sqx-custom-block` cuando falta algo. No arranca SQX y no gasta CPU: `vocabulary.py:87-88` sólo lee tres XML e `install.py` exige el worker parado.

### 1.2 La decisión, en tres preguntas

| # | Pregunta | Quién la contesta | Qué pasa si «no» |
|---|---|---|---|
| a | ¿La **lógica exacta** que describió el dueño ya está como bloque? | `vocabulary <término>`, y el criterio de una persona/agente: el texto de pantalla y la ayuda de SQX (`help`) dicen si un bloque prueba un **estado** o una **transición** | Se autora el bloque (b) |
| b | ¿El bloque nuevo es importable y no tiene fallos de forma? | `validate.py` (+ `assess.py`) de la skill | Se corrige el bloque concreto que falla |
| c | ¿El bloque está en **las dos** instalaciones que construyen, y un grupo lo alcanza si hace falta? | `install.py` + `vocabulary --diff custodian` | Se reinstala; sin esto la plantilla falla en silencio |

El paso **no** decide si la idea es buena (paso 1), ni si el bloque funciona de verdad (paso 4 en adelante, con una construcción), ni qué bloques puede sortear una construcción (paso 6, paletas).

### 1.3 Regla de oro de la autoría

`tools/sqx-lab/overlays/sqx-custom-block.md` (pegado encima del texto del fabricante) fija cuatro cosas: preguntar la lógica siempre (regla 11); **lo que dijo el dueño, exactamente** — si falta, se autora, nunca se aproxima con «el bloque más parecido»; lo que él nombró es fijo y los periodos/desviaciones que no nombró quedan como parámetros optimizables; y escribir también el bloque opuesto con `oppositeBlockKey`.

## 2. Fundamentos: los conceptos que hacen falta (sin estadística)

### 2.1 Bloque, clave y texto de pantalla

Un **bloque** es una pieza con nombre que el constructor de SQX puede colocar en una estrategia. Cada uno tiene una **clave** (`key`), que es lo que referencian los grupos y las plantillas, y un **texto de pantalla** (`display`) con huecos tipados entre almohadillas. Ejemplo nativo (`sqx/blocks/taxonomy.yaml`, `ADXCrossUp`): `ADX(@Chart@#Period#)[#Shift#] crosses above #Level#` — tres huecos que hay que rellenar: periodo, desplazamiento y nivel.

Hay tres **tipos**:

- **Condición** (`returnType="boolean"`): verdadero o falso en cada vela. Es lo que forma una regla de entrada o de salida.
- **Valor / indicador**: un número o precio (una media, el ATR). Se compara con otra cosa.
- **Nivel de precio** (`type="Price level"`, `returnType="price"`): un precio que usa una orden stop o límite. Con órdenes a mercado no se usa (regla 15).

### 2.2 Nativo y propio (custom block)

Los **nativos** (849) vienen con SQX, en `internal/web/SQWIZARD/branding/global/config.xml` (`vocabulary.py:13`, `native_blocks`). Los **propios** (179 hoy) los escribe el dueño y viven en `user/settings/customBlocks.xml` (`vocabulary.py:14`). Su clave empieza por `CBlock_` — así distingue un grupo un bloque propio de uno nativo (docstring de `vocabulary.py:50-51`).

Un bloque propio se **compone de piezas nativas**. El del ejemplo (apartado 3.9) es `CrossesAbove( Close, KeltnerChannel(periodo, desviación).Upper )`: un operador de comparación de SQX con el cierre a un lado y el indicador al otro. Lo que el propio bloque añade es: el nombre, la ayuda, qué parámetros quedan expuestos (aquí `#Period1#` y `#Double1#`, optimizables) y cuál es su **opuesto**.

### 2.3 Estado y transición — el matiz que decide si «ya existe»

Una pequeña cuenta a mano. Cierres de cuatro velas: 10, 11, 12, 13; banda superior fija en 11,5.

| vela | cierre | ¿cierre > banda? (estado) | ¿cruza por encima? (transición) |
|---|---|---|---|
| 1 | 10 | no | no |
| 2 | 11 | no | no |
| 3 | 12 | **sí** | **sí** (venía en 11, ≤ 11,5) |
| 4 | 13 | **sí** | no (ya venía por encima) |

El estado dispara en dos velas; la transición, en una. Un backtest abre muchas más operaciones con el estado. Para el dueño en el ejemplo, «cruza» era el cruce (transición), y el bloque nativo más parecido, `KCBarClosesAboveUpper`, es el estado. Por eso se autoró uno nuevo (`brief.md`, «Por qué hay que autorar el bloque»). **Esta distinción rara vez la dice el nombre**; la dice el `help` de SQX, que `vocabulary.py:58` guarda por eso.

### 2.4 Grupo aleatorio

Un **grupo aleatorio** es un menú del que el constructor saca **un** elemento por estrategia para rellenar **un** hueco. Una plantilla no referencia bloques sino grupos (`knowhow/authoring/holes-groups-randomcondition.md`). Tipos: **Condición** (elementos booleanos) y **Valor** (precios o números). Dos modos de llenarlo (SKILL de `sqx-random-group`): **híbrido** (los elementos son referencias a bloques propios `CBlock_*`, el caso común) e **inline** (reglas o comparaciones sueltas hechas con átomos nativos).

Tres situaciones de un hueco de la plantilla:

- **Hueco libre** (`#Group#` vacío): sortea las 500 condiciones. Un grupo sólo lo estrecha.
- **Hueco atado a un grupo:** sortea sólo ese grupo e **ignora** la paleta del paso 6 (`sqx/blocks/palettes/*` lo advierte en su cabecera).
- **Bloque fijo:** parte del esqueleto; no se estrecha desde ningún sitio.

Un grupo de **un solo** elemento fija la primera elección pero SQX puede cambiar el hueco más tarde por mutación: en `Test_USDJPY_donchianUpperCrossUp_M30`, 197 de 200 estrategias llevaban el bloque (`knowhow/authoring/one-item-group-can-drift.md`).

### 2.5 Dónde se guarda y qué es «instalar»

Son dos ficheros por instalación: `user/settings/customBlocks.xml` (bloques) y `user/settings/blockGroups.xml` (grupos). **Instalar** es escribir una entrada en ellos con la instalación **parada**. `sqcli` nunca los reescribe; la interfaz gráfica sí, al salir, y por eso una escritura con la instalación encendida se pierde sin avisar (`sqx/blocks/README.md`).

### 2.6 Qué NO entra en este paso: taxonomía y paleta

`sqx/blocks/taxonomy.yaml` (5.087 líneas, 767 filas) etiqueta cada bloque que el constructor puede sortear con siete pesos (0 a 3) por familia de idea: `breakout`, `mean_reversion`, `trend`, `momentum`, `volatility`, `pattern`, `session` (`taxonomy.py:23`). Una paleta (`palette.py`) combina esas etiquetas con una lista a mano. Es el **paso 6**. Aquí sólo importa por dos razones: la taxonomía se **regenera** de las instalaciones (`python3 -m sqx.blocks.taxonomy`) y un bloque propio nuevo **no entra** hasta que se regenera, y entonces entra sin etiqueta hasta que alguien lo etiqueta; y la negativa de la regla 15 se aplica sobre ella. «Un bloque sin etiquetar no entra en ninguna paleta» (cabecera del YAML).

---

## 3. Lo que hace, pieza a pieza

### 3.1 Leer el vocabulario — `vocabulary()`

`vocabulary.py:79`. Tres XML, nada más. Devuelve `{install, native, custom, groups}`.

- `native_blocks` (`:19-40`): recorre `config.xml` → `<Blocks>` → secciones → categorías → ítems; guarda sección, categoría, tipo devuelto, texto de pantalla y la ayuda de SQX.
- `custom_blocks` (`:43-59`): de `customBlocks.xml`, categoría, tipo, texto de pantalla (o el nombre si no hay) y ayuda.
- `groups` (`:62-76`): de `blockGroups.xml`, nombre, tipo y la lista de claves de sus ítems.

**Hoy** ( `recuento.py` ): conductor y custodio — 849 nativos, 179 propios, 25 grupos, 0 referencias muertas; maestro — 850 nativos, 171 propios, 22 grupos, y **2 referencias muertas** ( `keltnerUpperCrossUpSignal` apunta a `CBlock_CloseCrossesAboveKCUpper` y `emaCloseCrossUpSignal` a `CBlock_CloseCrossesAboveEMA` , que el maestro no tiene). Son hechos del maestro, que es del dueño: no los toco ni propongo nada.

### 3.2 Buscar «¿existe?» — `find()`

`vocabulary.py:110-126` . Minúsculas; devuelve cada bloque, nativo o propio, cuya **clave** o **texto de pantalla** contiene el término; ordena por clave y añade qué grupos lo contienen. No busca en `category` ni en `help` .

Dos ejemplos reales del conductor:

| consulta | resultado | lectura |
|---|---|---|
| `keltner` | 18 nativos ( `KCBarClosesAboveUpper` , …, `KeltnerChannel` , `MTKeltnerChannel` ), **0 propios** | engañosa: existen dos propios de categoría `Keltner` |
| `KCUpper` | `CBlock_CloseCrossesAboveKCUpper` (propio, `pooled by: keltnerUpperCrossUpSignal` ), `KCUpperFalling` , `KCUpperRising` | aquí sí aparece |

Para decidir «ya existe» hay que mirar además la ayuda y la categoría en `taxonomy.yaml` ( `grep -n 'Keltner' sqx/blocks/taxonomy.yaml` ) o en el `catalog.md` de la skill. El código no lo hace por ti.

### 3.3 Saber si algo lo alcanza — `pooled_by()`

`vocabulary.py:95-107` : los grupos que contienen la clave. La consola lo imprime como «pooled by:» o, si no hay ninguno, como «NO GROUP POOLS IT — unusable in a template» ( `:206` ). Hoy no están en ningún grupo **130 de los 179 propios y 808 de los 849 nativos** (41 nativos y 49 propios sí lo están; los 41 nativos son claves de los cuatro grupos de valor). Lo importante: para un propio, «sin grupo» significa que **sólo** lo alcanza un hueco libre o un bloque fijo; para un hueco atado a un grupo, no existe.

### 3.4 El inventario — `report()` y `usable_groups()`

`:129-147` y `:166-184` . Cuenta nativos, propios y grupos, parte los grupos en de condición / de valor / vacíos, y avisa de dos cosas: un grupo **vacío** («EMPTY, unusable»; sin error, una plantilla atada a él construye y no sortea nada) y la **ausencia** de grupos de valor («every stop-entry shape is ungenerable here»). Hoy: `RandConditions` vacío, 4 grupos de valor ( `BollingerBands_Lower` , `BreakoutRand` , `GroupMAs` , `RandomPrices` ).

### 3.5 Comparar instalaciones — `missing()`

`:150-163` : claves de bloque propio y nombres de grupo que el destino no tiene frente a la referencia. Es la comprobación de que la instalación que **construye** tiene lo que la que **autora** escribió.

| comando | resultado hoy |
|---|---|
| `--diff custodian` (con conductor de referencia) | 0 bloques y 0 grupos que falten |
| `--diff master` | faltan 8 bloques ( `CloseCrossesAbove/Below` de `DCUpper` , `EMA` , `HMA` , `KCUpper` / `KCLower` ) y 3 grupos ( `crossAboveHMA_v1Signal` , `donchianUpperCrossUpSignal` , `emaCloseAboveSignal` ) |

Que el maestro no los tenga es por la regla 3, no un fallo. Compara **nombres**, no contenido: dos bloques con la misma clave y distinta definición pasan el `--diff`.

### 3.6 Escribir una instantánea — `--snapshot`

`vocabulary.py:219-223` : escribe el inventario completo en `AlgoData/templates/vocabulary/<instalación>-<fecha>.json` . Hoy sólo existen los de **2026-09-22** ( `SQX` , `SQX_w1` , `SQX_w2` ); nadie la ha vuelto a correr.

### 3.7 Instalar — `install.py`

Entrada: `python3 -m sqx.blocks.install <xml> [--role conductor|custodian|master]` (por defecto el conductor). Pasos, en orden:

1. **Elige el rol** ( `install.py:94-98` ). `master` → `SystemExit` («the master is the owner's install», `:99-101` ).
2. **Rechaza si el puerto del worker responde** ( `:102-105` ): `worker.listening(port)` intenta conectar a `127.0.0.1:<puerto>` ( `core/worker.py:162` ). Es la única comprobación de «instalación en marcha». No mira el fichero `OWNER` ni si otra sesión la tiene reservada (eso queda a la disciplina de la regla 3).
3. **Decide el fichero** según el primer hijo del XML ( `store_of` , `:46-53` ): `Item` → `customBlocks.xml` , clave `key` ; `Group` → `blockGroups.xml` , clave `name` . El tipo del primer hijo decide almacén y atributo-clave para **todos** los hijos ( `:75-82` no filtra por etiqueta): un `<Group>` dentro de un fichero que empieza por `<Item>` se escribe en `customBlocks.xml` con clave `None` , sin error (probado por mí en un directorio temporal con la función `install()` ).
4. **Hace copia** ( `backup` , `:30-43` ): `<fichero>-backups/<milisegundos>.xml` . Hoy `SQX_w1` guarda 14 copias de `customBlocks` (15 MB) y 2 MB de `blockGroups` ; nada las poda.
5. **Añade o sustituye** ( `install` , `:56-86` ): por cada entrada, si la clave ya está se quita la vieja y se añade la nueva **al final** ( `:77-82` ); si no, se añade. **No impide duplicados**: `present` se calcula una vez antes del bucle ( `:72` ) y no se actualiza al añadir. Un lote con la misma clave nueva dos veces la añade dos veces (probado: `added ['B','B']` ); con la clave ya instalada y repetida en el lote, el segundo `root.remove` lanza `ValueError` , después de hacer la copia. La única comprobación de duplicados dentro del lote es la 2 de `validate.py` , si se corre. Hoy hay 0 claves repetidas en el XML crudo de las tres instalaciones (contado sobre el XML, no sobre `vocabulary()` , que colapsa claves repetidas en un diccionario). Además, `replaced` no distingue la misma definición de una distinta: las plantillas y grupos que apuntan a esa clave pasan a usar la lógica nueva sin que nada lo registre, y la escritura ( `:84-85` ) no es atómica (directa sobre el fichero, sin temporal ni renombrado).
6. **Reescribe todo el fichero** con `ElementTree.indent` ( `:84-85` ), es decir, re-sangra todo el almacén de 1,3 MB.

7. **Imprime** lo añadido/reemplazado, cuántas entradas hay y dónde está la copia.

Corolario operativo (`overlays/sqx-custom-block.md`, pasos 3-4): se instala **en las dos**, conductor y custodio, y se cierra con `--diff custodian` más `bin/sqx-lab-install.sh` (que reconstruye los catálogos de las skills).

### 3.8 Lo que dice la skill antes de instalar — `validate.py` (bloques)

`~/.claude/skills/sqx-custom-block/engine/validate.py`. Siete comprobaciones; las líneas son de ese fichero.

| # | Comprueba | Línea | Qué evita |
|---|---|---|---|
| 1 | XML bien formado | `:157-160` | `<`, `>`, `&` sin escapar en `display` o `help` |
| 2 | Sin claves duplicadas en el lote | `:66-72` | dos bloques con la misma clave |
| 3 | Pares opuestos simétricos: si A apunta a B, B apunta a A | `:73-86` | un espejo roto (largo/corto) |
| 4 | Todo `#ParamN#` usado en `<Contents>` está declarado arriba | `:88-100` | un parámetro fantasma |
| 5 | Claves de `<Param>` únicas dentro del bloque | `:102-105` | colisión de parámetros |
| 6 | El nombre **no** termina en `_<dígitos><letras>` (`_144Native`, `_2v3`) | `:107-112` | AlgoWizard recorta ese sufijo y rompe el enlace con el opuesto |
| 7 | Con catálogo: todo átomo de varias salidas lleva `#Line#` y ningún átomo `talib_*` | `:114-131` | el `NullPointerException` del Stockpicker al construir |

Si se da un `--catalog` que no existe y no hay otro de repuesto, falla en duro (`:173-184`). Sin `--catalog` la comprobación 7 se salta con una nota («check 7 SKIPPED») y el resultado final sigue siendo «ALL CHECKS PASSED», código 0 (`:189-194`).

Lo que **no** comprueba: que el indicador exista en esta instalación (eso lo hace el gestor de catálogo al emitir, no el validador), que el opuesto ya instalado cuente (la 3 exige que el opuesto venga **en el mismo lote**: si no viene, falla aunque ya esté instalado; para un grupo es sólo un aviso), ni compara con el almacén instalado (una clave ya instalada con otra lógica se reemplaza sin más), que el bloque no use reloj, que sea una transición o un estado, ni que la `key` anidada de un indicador sea la del catálogo.

### 3.9 Ejemplo de principio a fin — `CBlock_CloseCrossesAboveKCUpper`

**La idea del dueño** (`brief.md`, 2026-09-22): «entrar en largo cuando el cierre cruza por encima de la banda superior del canal de Keltner», con una condición fija y una aleatoria libre.

**Paso a paso**, con lo que consta en disco:

1. **¿Existe?** La categoría nativa `Keltner Channel` trae 16 condiciones y ninguna es un cruce sobre el cierre. La más cercana, `KCBarClosesAboveUpper`, es un estado (`brief.md`). El dueño eligió «cruce, no estado» de forma explícita el 2026-09-22.

2. **Se autoró** `blocks.xml` (6.070 bytes, 2 bloques): `CBlock_CloseCrossesAboveKCUpper` (el pedido) y `CBlock_CloseCrossesBelowKCLower` (su espejo, que «existe para que el largo tenga un opuesto que resolver»; los dos se apuntan entre sí). Atributos del primero: `type="Condition"`, `returnType="boolean"`, `category="Keltner"`, `oppositeBlockKey="CBlock_CloseCrossesBelowKCLower"`. Contenido: `CrossesAbove(Close[@Chart@ 1], KeltnerChannel(@Chart@ #Period1# , #Double1#).Upper[1])`. Parámetros expuestos: `#Period1#` (`KC_Period`, por defecto 20, rango 2–220) y `#Double1#` (`KC_Deviation`, por defecto 1,5, rango 0,1–7).
3. **Comprobación.** Corrí hoy el `validate.py` de la skill sobre ese fichero con su catálogo: las 7 pasan, «ALL CHECKS PASSED». El `assess.py` no calificó nada (apartado 5.2).
4. **Instalación.** `python3 -m sqx.blocks.install deps/blocks.xml --role conductor` (el campo `produced_by` de `manifest.json` registra **sólo** el conductor; el custodio lo tiene por una vía que no consta). Hoy está en `SQX_w1` y `SQX_w2` — idéntico al XML autorado salvo una línea en blanco que añade el re-sangrado — y **no** en `SQX`. `vocabulary --diff custodian` da cero faltantes.
5. **Un grupo de un solo elemento** `keltnerUpperCrossUpSignal` (`groups.xml`, id `c6e3f1d4-84b3-535f-abdb-16604a435f85`) con ese bloque y los dos parámetros marcados `generate="random"`; el 2026-09-26 se pasó de «bloque fijo» a «grupo de un solo ítem» para que el constructor sortee los periodos (antes quedaba congelado el mismo en todas las estrategias; `manifest.json`, campo `rebuilt`).
6. **Prueba real.** Construcción en `SQX_w2`, proyecto `smoke_keltnerUpperCrossUp`, XAUUSD H1, ventana 2008-01-01 a 2017-12-31: **30 estrategias en 29 s y las 30 llevan el bloque fijo** (`build_confirmed`). Tras el cambio a grupo, humo USDJPY H1: 15 de 15 con periodos variados. Los costes del humo eran «provisionales, defaults de SQX» y no se leyeron resultados, sólo que el bloque estuviera.
7. **Hoy** la plantilla está retirada, y los bloques siguen instalados.

---

## 4. Lo que se niega, lo que se valida, y dónde

Este paso no tiene umbral: tiene **negativas** y **comprobaciones**. Tabla completa, con dónde vive cada una y si se aplica en el paso 2.

| Negativa o comprobación | Dónde | ¿Se aplica en el paso 2? | Por qué existe |
|---|---|---|---|
| No escribir en el maestro | `install.py:99-101` | **Sí** (negativa dura) | regla 3: su interfaz es del dueño |
| No escribir con el worker encendido | `install.py:102-105` | **Sí** (sólo por puerto) | SQX reescribe `user/settings` al salir (regla 4) |
| Una clave ya presente se reemplaza, no se duplica | `install.py:76-82` | **Sí** | SQX lee la primera coincidencia; un duplicado es invisible |
| Copia de seguridad antes de escribir | `install.py:107-108` | **Sí** | volver atrás |
| 7 comprobaciones de bloque | `validate.py` de `sqx-custom-block` | **Sí, si alguien la corre** | importable y sin sufijos que rompan el opuesto |
| 8 comprobaciones de grupo (no vacío, tipo coherente, ítems planos, `CBlock_*` resuelto contra la instalación viva) | `validate.py` de `sqx-random-group` (cabecera, `:7-27`) | **Sí, si alguien la corre** | un grupo con una clave que no resuelve es «un pool muerto» |
| `shift 0` rechazado en átomos de precio/indicador; átomos `talib_*` rechazados | `Catalog.atom()` en `emit.py:156-196` de la skill (con salidas `allow_shift0` y `allow_talib`) | **Sólo al emitir con la API Python de la skill**; un XML escrito a mano o clonado no pasa por ahí (`talib` lo cubre además la comprobación 7 con catálogo; `shift 0` sólo `assess.py`, que no corrió sobre el ejemplo) | mirar la vela en formación es mirar el futuro |
| `assess.py`: CRÍTICO (repintado, `shift` 0, umbral fuera del rango) / AVISO / INFO | skill, paso 5b | **Sí, pero sólo para bloques con `categoryType="Custom blocks"`** | calidad, no sólo importabilidad |
| **Regla 15: sin bloques de reloj ni calendario** | `buildcensus.py:14` (regex) y `buildingblocks.py:62` | **No: paso 6**, al aplicar la paleta a la tarea Build | se pide, se deja apagado y se imprime «regla 15: N pedidos y dejados apagados» |
| **Regla 15: sin niveles stop/limit** (`stopLimitBlocks`) | `buildcensus.py:18` y `buildingblocks.py:62` | **No: paso 6** | las órdenes son a mercado; sólo valen con stop o límite |
| Máximo 2 condiciones de entrada y 2 de salida (regla 14) | `sqx/CLAUDE.md`; dónde se cuenta en código no lo verifiqué | **No: paso 6** | `sqx/CLAUDE.md` «How many conditions…» |
| Una dirección por plantilla (regla 13) | `sqx.projects.builder` (`builder.directions`, según el `CLAUDE.md` raíz; no lo leí) | **No: paso 3 y siguientes** | una plantilla larga y corta a la vez no es una idea |

**Lectura sencilla:** en el paso 2 el sistema impide dos cosas (escribir en el maestro y escribir con el worker encendido) y recomienda el resto. Una violación de la regla 15 en un bloque recién escrito no la ve el paso 2; el paso 6 la ve solo si una paleta enciende ese bloque y la regex lo reconoce (apartados 5.3 y 5.10).

**Lo que nada comprueba en ningún sitio** (hecho comprobado leyendo los tres ficheros): que el bloque nuevo haga lo que dijo el dueño — que sea una transición y no un estado, que el opuesto sea realmente el espejo, o que el `shift` de un cruce sea el de la intención. Eso lo cierra, o no, la construcción de humo (apartado 3.9, punto 6) y la lectura humana.

---

## 5. Lo que no cuadra o no está medido

### 5.1 `find()` no ve los bloques propios cuyo texto no contiene el término

`vocabulary.py:124` compara con `key` y `display`. Para casi todos los propios `display` es igual al nombre (por ejemplo `CloseCrossesAboveKCUpper`), así que `keltner` no los encuentra: 18 nativos, 0 propios. El flujo de `sqx-strategy-template` y el overlay de `sqx-custom-block` mandan «comprueba con `vocabulary <término>`» (`tools/sqx-lab/overlays/sqx-custom-block.md`, punto 2; `overlays/sqx-strategy-template.md:29`), y un agente que se fíe de un «0 propios» puede **reautorar** lo que ya existe. Ese segundo bloque se instalaría con la misma clave, **reemplazando** al anterior (apartado 3.7, paso 5) y sin aviso. No medido: cuántos de los 179 propios no contienen en su texto el nombre del indicador (no lo calculé).

### 5.2 `assess.py` no califica los bloques que el propio flujo produce

`assess.py:271` filtra `categoryType == "Custom blocks"`; si no hay ninguno, imprime «no custom blocks found» y devuelve 0 (`:272-274`), incluso con `--strict`. El `blocks.xml` del ejemplo (autorado y con las 7 comprobaciones pasadas) no lleva ese atributo en su ítem raíz; resultado: código de salida 0, cero bloques calificados. Es un portón que sólo se enciende para el formato que emite `make_block` (`grammar.py:383,405`). Un bloque escrito a mano, copiado o clonado de otro lo salta. Los usos de la skill pueden ser distintos; sólo he comprobado este fichero.

### 5.3 La lista de «bloques de reloj» es una expresión regular y deja fuera familiares

`buildcensus.py:14`: `^(Current|Bar)(Hour|Minute|DayOf|WeekOf|Month|Time)|^IsMonth` sobre la parte de la clave tras el último punto. Atrapa 29 nativos de «Bar And Time». Pero sobre las **767 filas de la taxonomía**, ocho claves con aspecto de reloj o sesión quedan fuera:

| clave | origen | qué es (por su nombre/forma) | la encienden… |
|---|---|---|---|
| `CBlock_TimeBreakoutLong`, `CBlock_TimeBreakoutLong_2` | propio | ruptura de un rango horario (`TimeBreakoutLong(#Int2#, #Int3#)`, dos enteros); `_2` es en realidad `TimeBreakoutShort` (su `display`) | `ruptura_base` |
| `SessionHigh`, `SessionLow` | nativo | máximo/mínimo de la sesión | `ruptura_base`, `sesion_base` |
| `SessionOpen` | nativo | apertura de la sesión | `timeRangeBreakout`, `sesion_base` |
| `SessionClose` | nativo | cierre de la sesión | ninguna |
| `VolumeProfileCustomHours`, … `MultiSession` | nativo | perfil de volumen por horas | ninguna |

Calculado por mí con `palette.resolve` sobre las 20 paletas del directorio (`recuento.py`): ninguna de las paletas `*_v3` ni las específicas de un activo enciende estos ocho. Las tres que sí son anteriores. **Qué es «reloj» aquí lo decide el dueño**: la regla dice «cualquier bloque que lea el reloj o el calendario»; `SessionHigh` lee la sesión (una ventana horaria) y `CBlock_TimeBreakoutLong` lee horas. Es una pregunta del tipo de la regla 11, no una corrección que yo haga (apartado 6, decisión 3). Fuera de la regex pero ausentes de la taxonomía, y por tanto inalcanzables para el constructor: `FirstWeekOfMonth`, `LastWeekOfMonth`, `BarDate`, `CurrentDate`.

**Matiz de 5.3 (revisión del Council):** la regex se aplica a la parte de la clave tras el último punto y empieza por `^(Current|Bar)` (`buildcensus.py:14,36`); una clave `CBlock_*` nunca empieza así. No es solo el caso de `CBlock_TimeBreakoutLong`: ningún bloque propio puede ser clasificado como reloj. Las demás claves de «Bar And Time» fuera de la regex (`IsBarOpen`, `AlwaysFalse`, `AlwaysTrue`, `CurrentBar`) no son reloj.

### 5.4 El instalador no valida; la validación depende de acordarse

`install.py` no comprueba el prefijo `CBlock_`, ni que haya al menos un ítem (un XML vacío da `IndexError` en `store_of`, `:53`), ni que un grupo apunte a bloques que existen. Y la `key` anidada de un indicador mal puesta pasa tanto `install` como `sqx.templates.build` y sólo falla al construir: `XmlStrategyException … Cannot find block 'HMA'` por cada candidato, con `Failed` en 0 y el tope de minutos gastándose sin generar nada (`knowhow/authoring/cloned-custom-block-native-key.md`, □ 2026-09-26). Ninguna de las 7 comprobaciones de `validate.py` lo ve.

### 5.5 «Sustituir en su sitio» no es en su sitio

`sqx/blocks/README.md` y la cabecera de `install()` dicen «replaced in place». El código **quita y añade al final** (`install.py:77-82`). Importa poco al constructor, pero un `git diff` o una comparación del almacén muestra un reordenamiento total del bloque reinstalado, y mezcla en la lectura el cambio real. Además, el fichero entero se re-sangra en cada instalación (`:84`).

### 5.6 Un grupo reinstalado con el mismo nombre puede recibir un `id` nuevo

La skill de grupos emite un UUID aleatorio **por defecto** (`groups.py:353`: `group_id or str(uuid.uuid4())`; `keltnerUpperCrossUpSignal` lleva un id fijo de versión 5, idéntico en `groups.xml` y en las dos instalaciones, así que su reinstalado lo conserva), y el instalador sustituye por **nombre** (`STORES`,

`install.py:16`). Una plantilla referencia el grupo por su id y lleva dentro una copia (en `template.sqx`, `strategy_Portfolio.xml` contiene el grupo entero). **No he comprobado** qué hace SQX si el id de la copia embebida y el instalado difieren; el overlay `sqx-random-group.md` dice que instalar grupos por fichero «no está probado por una construcción»; el manifest de `keltnerUpperCrossUp` registra, en cambio, un humo USDJPY con el grupo (15/15), sin constar con qué comando se instaló el grupo. El texto del overlay está probablemente desfasado.

### 5.7 Texto y números desfasados

- `AlgoData/manual-fuentes/21-vocabulario.md` da «849 native + 171 own blocks, 20 groups», 15 grupos de condición. Hoy: 179 propios, 25 grupos, 20 de condición. (Los 171 son los del maestro de hoy.)
- El overlay (`overlays/sqx-custom-block.md`) dice que los catálogos se construyen contra el conductor (`~/.sqx-lab` → `SQX_w1`). Hoy `~/.sqx-lab/sqx-install.txt` contiene `SQX_w2` (el custodio) y los dos `catalog.json` apuntan a `SQX_w2`. Las dos instalaciones son idénticas ahora, así que no cambia nada; si divergieran, el catálogo seguiría al custodio.
- El catálogo de átomos de `sqx-custom-block` (`catalog.json`) tiene **235 átomos** (233 leídos de `config.xml` + 2 sintetizados del registro; **ninguno propio**: ninguna clave empieza por `CBlock_`): es un índice de valores, **no** del vocabulario de 849 bloques. Sirve para emitir; no responde «¿existe?».
- Las instantáneas de `AlgoData/templates/vocabulary/` son de 2026-09-22.
- `knowhow/authoring/block-vocabulary.md` dice 171 propios (fecha 2026-09-22).

### 5.8 El mensaje «unusable in a template»

`vocabulary.py:206` y `:181` usan «unusable» para lo que en realidad significa «ningún grupo lo contiene». Para un bloque que va a un hueco libre es falso (hecho de `holes-groups-randomcondition.md`). Para bloques nativos aparece siempre, de modo que acaba siendo ruido.

### 5.10 Otros fallos silenciosos que hay (descritos, sin propuesta)

- **`--diff` es unidireccional y por nombre** (`vocabulary.py:162-163`): dice lo que falta en el destino, no lo que sobra, ni compara el contenido de un bloque ni los ítems de un grupo.
- **Conductor y custodio son dos órdenes sin transacción**: el primero puede quedar instalado y el segundo no llegar a hacerse.
- **La escritura del almacén no es atómica** (`install.py:84-85`): un corte durante `tree.write` deja el fichero truncado; la copia previa solo sirve si alguien la restaura a mano.
- **Reemplazar no avisa de qué lo usa**: `replaced` no distingue la misma definición de otra, ni lista los grupos o plantillas que apuntan a esa clave.
- **Lote con claves repetidas**: duplica (clave nueva) o falla a medias tras la copia (clave ya instalada); ver 3.7, paso 5.
- **La taxonomía va por detrás de la instalación** (8 propios sin fila), ver punto 8 de la sección 0.
- **El `display` puede engañar**: `CBlock_TimeBreakoutLong_2` es `TimeBreakoutShort`; el nombre de la clave no dice la dirección.

### 5.9 Lo que no he podido verificar

- Que `CrossesAbove` de SQX compara el cierre de la vela 1 con la banda de la vela 1 **y** de la 2 (la definición viene dentro de SQX; no leí su código). El `help` del propio bloque describe el cruce, pero es texto del autor.
- Que los 14 respaldos de `customBlocks-backups` correspondan a instalaciones de este paso (no hay registro de quién los hizo).
- Qué ocurre si un worker está arrancándose (puerto aún sin escuchar) cuando se instala: la comprobación es sólo por puerto (`connect_ex`, 0,5 s), no por proceso ni por el fichero `OWNER`; `core.worker.holding(install)` (PIDs por instalación) existe en la misma librería y el instalador no lo usa.

---

## 6. Cierre: decisiones que te quedan

Cada una con lo que gana y lo que cuesta. Sólo describo: ninguna la he tomado.

1. **¿Qué significa «ya existe»?** Hoy: subcadena en clave o texto de pantalla (`vocabulary.py:124`). - *Ganar:* ampliar la búsqueda a categoría y ayuda evita reautorar. Un bloque propio de categoría `Keltner` saldría con `keltner`. - *Coste:* más ruido en las respuestas (ayudas largas mencionan indicadores de pasada) y decidir si «existe» exige además leer a mano si es estado o transición. - *Sin hacerlo:* el riesgo concreto es reescribir con la misma clave un bloque que ya está y que ya llevan plantillas.
2. **¿Debe el instalador validar él mismo?** Hoy no. - *Ganar:* un fallo de forma no llega nunca a `customBlocks.xml`; se cierra el hueco de 5.4 en el sitio donde ya se pasa siempre. - *Coste:* el instalador pasa a depender de la skill (hoy el repositorio no depende de ella) o a duplicar sus siete reglas.
3. **¿Qué es un bloque de reloj para la regla 15?** Hoy lo define una regex y quedan fuera `CBlock_TimeBreakoutLong(_2)` (propio), `SessionHigh/Low/Open/Close` y `VolumeProfileCustomHours*`. - *Ganar:* una lista que coincide con lo que dices; las paletas antiguas `ruptura_base`, `sesion_base` y `timeRangeBreakout` dejarían de encender esos bloques. - *Coste:* la familia `session` (una de las siete de la taxonomía) pierde sus bloques característicos; si «sesión» es una familia legítima y «reloj» sólo lo suyo literal, la regla debe decir cuál es cuál. Es una pregunta de la regla 11.
4. **¿Dónde se aplica la regla 15: sólo al aplicar la paleta, o también al autorar?** - *Ganar:* un bloque de reloj o de nivel stop/limit no entra en el almacén, y deja de depender de que cada paleta lo apague. - *Coste:* un bloque de reloj legítimo para una plantilla con idea de sesión (fijo, no sorteado) quedaría imposible de instalar.
5. **¿El almacén de las dos instalaciones se compara por nombre (hoy) o por contenido?** `--diff` no ve la diferencia entre dos definiciones distintas de la misma clave. Ganar: detectar un worker con una versión vieja; coste: una comparación de XML que ignore el re-sangrado.
6. **¿Se corrige el texto desfasado?** El capítulo 21 del manual, el overlay (conductor/custodio y «no probado») y la ficha `block-vocabulary` dicen cifras del 2026-09-22; el coste es sólo trabajo documental; el beneficio, que quien lee el PDF vea 849 + 179 y 25 grupos. Dime si quieres que lo haga otra sesión.

7. **¿`assess.py` debe calificar los bloques sin `categoryType="Custom blocks"`?** Es código del fabricante (`tools/sqx-lab/`); tocarlo lo aparta de la versión original (`tools/sqx-lab/LOCAL_PATCHES.md` lleva ese registro).